

Question 2. When the user runs a program in a text-based environment like the Python Shell, what determines the order in which things happen?

When the user runs a program in a text-based environment like the Python Shell, the order is determined by the sequence of the statements in the code. The only possible modifications are made by control flow items like loops and functions, which makes the program jump to different parts of the code.

Question 3. Briefly describe each of the following tkinter widgets:

- a. Label** → Displays static text or images on the screen
- b. Entry** → Allows the user to input text into the GUI
- c. Button** → Serve as buttons to trigger certain actions or events
- d. Frame** → Used as a container to help group and organize other widgets

Question 4. How do you create a root widget?

You can create a root widget by instantiating the main window class, by doing `root = Tk()`. This creates an object that serves as the base container and starts the interpreter's event loop to help manage the application.

Question 5. What does the tkinter module's `mainloop()` function do?

The tkinter's module's `mainloop()` function runs the program (GUI) until the program is closed. This means that even if the program reaches the last line of code, the program will continue to run, leaving the application open and working.

Question 6. What does a widget's `pack()` method do?

The `pack()` method is like a manager that organizes widgets into blocks before placing them back into a parent container. It sizes and positions widgets based on the available space, stacking them vertically from top to bottom.

Question 7. If you create two Label widgets and call their `pack()` methods with no arguments, how will the Label widgets be arranged inside their parent widget?

The label widgets would be arranged vertically inside their parent widget, as the first label would be placed on top and the second label would be underneath it.

Question 8. What argument would you pass to a widget's `pack()` method to specify that it should be positioned as far left as possible inside the parent widget?

I would pass in the argument `side='left'` to the `pack` method, as this would tell the manager to place the widget on the left side of the remaining space within the parent container.